

Algorithms

Lecture #08

Syed Hamed Raza

Merge Sort

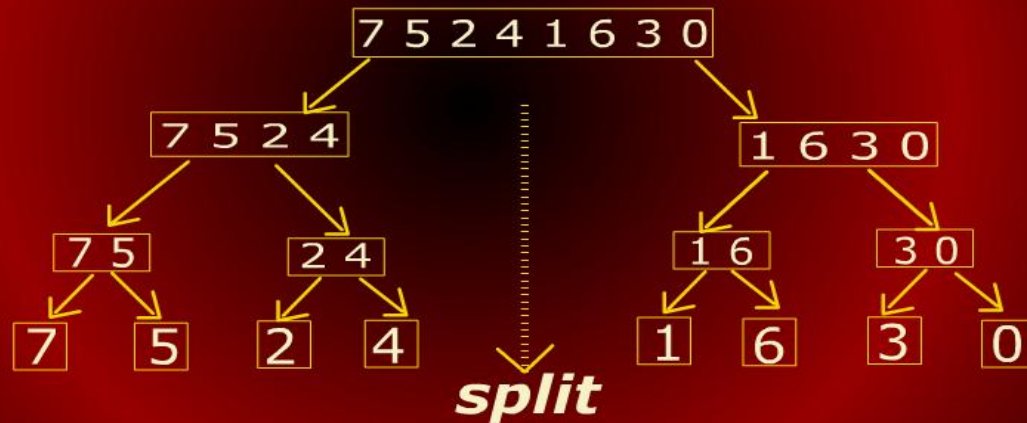
Merge Sort

- The dividing process ends when we have split the subsequences down to a single item.
- A sequence of length one is trivially sorted.
- The key operation is the combine stage which merges together two sorted lists into a single sorted list.
- Fortunately, the combining process is quite easy to implement.

Merge Sort

Merge sort

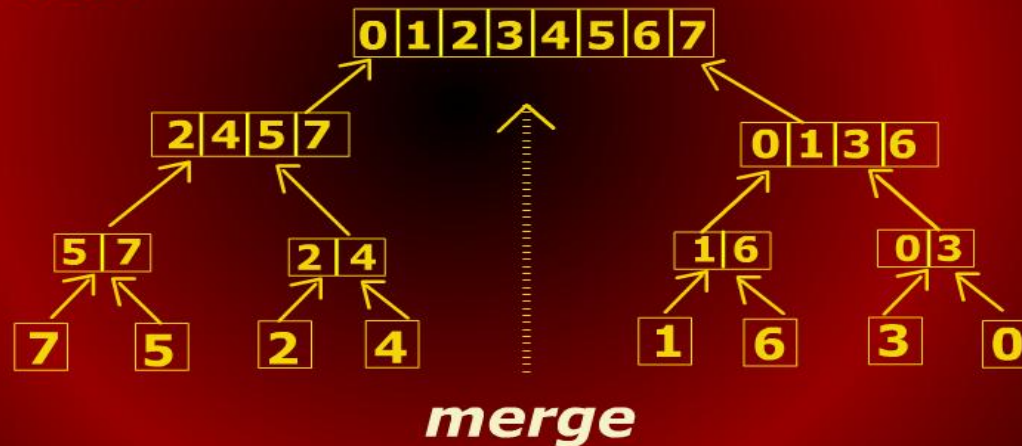
The divide phase: split the list top-down into smaller pieces



Merge Sort

Merge sort

The conquer phase: merge the smaller lists bottom up into larger pieces



Merge Sort Algorithm

Merge Sort Algorithm

MERGE-SORT(array A, int p, int r)

1 **if** ($p < r$)

2 **then**

3 $q \leftarrow (p + r)/2$

4 MERGE-SORT(A, p, q) // sort A[p..q]

5 MERGE-SORT(A, q+1, r) // sort A[q+1..r]

6 MERGE(A, p, q, r) // merge the two pieces

Merge Sort Algorithm

Merge Sort Algorithm

```
MERGE( array A, int p, int q, int r)
1  int B[p..r]; int i ← k ← p; int j ← q + 1
2  while (i ≤ q) and ( j ≤ r)
3  do if (A[i] ≤ A[ j])
4      then B[k++ ] ← A[i++ ]
5      else B[k++ ] ← A[ j++ ]
6  while (i ≤ q)
7  do B[k++ ] ← A[i++ ]
8  while ( j ≤ r)
```


Merge Sort Algorithm

Merge Sort Algorithm

```
3  do if ( $A[i] \leq A[j]$ )
4      then  $B[k++ ] \leftarrow A[i++ ]$ 
5      else  $B[k++ ] \leftarrow A[j++ ]$ 
6  while ( $i \leq q$ )
7  do  $B[k++ ] \leftarrow A[i++ ]$ 
8  while ( $j \leq r$ )
9  do  $B[k++ ] \leftarrow A[j++ ]$ 
10 for  $i \leftarrow p$  to  $r$ 
11 do  $A[i] \leftarrow B[i]$ 
```

Analysis of Merge Sort

Analysis of Merge Sort

- First, consider the running time of the procedure $\text{merge}(A, p, q, r)$.
- Let $n = r - p + 1$ denote the total length of both left and right sub-arrays, i.e., sorted pieces.
- The merge procedure contains four loops, none nested in the other.
- Each loop can be executed at most n times.
- Thus running time of merge is $\Theta(n)$

Analysis of Merge Sort

Analysis of Merge Sort

- Let $T(n)$ denote the worst case running time of MergeSort on an array of length n .
- If we call MergeSort with an array containing a single item ($n = 1$) then the running time is constant.
- We can just write $T(n) = 1$, ignoring all constants.

Analysis of Merge Sort

Analysis of Merge Sort

- For $n > 1$, MergeSort splits into two halves, sorts the two and then merges them together.
- The left half is of size $\lceil n/2 \rceil$ and the right half is $\lfloor n/2 \rfloor$.
- How long does it take to sort elements in sub array of size $\lceil n/2 \rceil$?

Analysis of Merge Sort

Analysis of Merge Sort

- The left half is of size $\lceil n/2 \rceil$ and the right half is $\lfloor n/2 \rfloor$.
- How long does it take to sort elements in sub array of size $\lceil n/2 \rceil$?
- We do not know this but because $\lceil n/2 \rceil < n$ for $n > 1$, we can express this as $T(\lceil n/2 \rceil)$.

Analysis of Merge Sort

Analysis of Merge Sort

- Similarly the time taken to sort right sub array is expressed as $T(\lfloor n/2 \rfloor)$.

- In conclusion we have

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n & \text{otherwise} \end{cases}$$

- This is called ***recurrence relation***, i.e., a recursively defined function.

Solving the Recurrence

Solving the Recurrence

Let's expand the terms.

$$T(1) = 1$$

$$T(2) = T(1) + T(1) + 2 = 1 + 1 + 2 = 4$$

$$T(3) = T(2) + T(1) + 3 = 4 + 1 + 3 = 8$$

$$T(4) = T(2) + T(2) + 4 = 8 + 8 + 4 = 12$$

$$T(5) = T(3) + T(2) + 5 = 8 + 4 + 5 = 17$$

...

Solving the Recurrence

Solving the Recurrence

...

$$T(8) = T(4) + T(4) + 8 = 12 + 12 + 8 = 32$$

...

$$T(16) = T(8) + T(8) + 16 = 32 + 32 + 16 = 80$$

...

$$T(32) = T(16) + T(16) + 32 = 80 + 80 + 32 = 192$$

Solving the Recurrence

Solving the Recurrence

What is the pattern here?

- Let's consider the ratios $T(n)/n$ for powers of 2:

$$T(1)/1 = 1$$

$$T(8)/8 = 4$$

$$T(2)/2 = 2$$

$$T(16)/16 = 5$$

$$T(4)/4 = 3$$

$$T(32)/32 = 6$$

- This suggests $T(n)/n = \log n + 1$
- Or, $T(n) = n \log n + n$ which is $\Theta(n \log n)$ (use the limit rule).

Solving the Recurrence

Analysis of Merge Sort

Design and Analysis of Algorithm

Merge Sort